IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce and Management

Final Project Report

on

Multi-Cloud Hybrid Strategy for Healthcare

Student Name: Adharv Shyam

Industry Mentor:

Mr. Sumit Kumar Shukla

Guided by:

Mr. Praveen

IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Table of Contents

SI No	Table of Contents	Page No
1.	Executive Summary	3
2.	Background	4
2.1	Aim	4
2.2	Technologies	5
2.3	Hardware Architecture	6
2.4	Software Architecture	7
3.	System	8
3.1	Requirements	8
3.1.1	Functional Requirements	8
3.1.2	User Requirements	8
3.1.3	Environmental Requirements	9
3.2	Design and Architecture	9
3.3	Implementation	10
3.4	Testing	12
3.5	Graphical User Interface (GUI) Layout	13
3.6	Customer Testing	13
3.7	Evaluation	14
4.	Snapshots of the Project	16
5.	Conclusions	22
6.	Further Development or Research	23
7.	References	24
8.	Appendix	26

1. Executive Summary

Multi-Cloud Hybrid Strategy for Healthcare is a backup-focused project designed to protect sensitive healthcare information through redundancy, integrity verification, and cloud failover. The system works with a simulated healthcare dataset that includes patient details, appointments, prescriptions, and laboratory reports, and it demonstrates how these records can be copied safely into more than one storage environment.

The backup workflow begins by reading the healthcare dataset from the source folder and creating a local backup copy. After the local copy is created, the system performs integrity verification using SHA256 hashing and file-size comparison. Once the local backup is validated, the same files are uploaded to Google Drive and AWS S3 so that off-site copies are maintained in separate cloud environments.

A major strength of the project is its reliability-oriented design. If one cloud destination fails, the system still has another cloud target available. If both cloud uploads fail, the failover logic ensures that the validated local backup remains available. This layered approach demonstrates how hybrid backup design can reduce operational risk and improve recoverability.

The project also includes a logging system and a terminal-based dashboard summary. These outputs make the backup process traceable and easier to monitor. Overall, the system provides a practical demonstration of multi-cloud backup principles in a healthcare context, where data continuity and trust are especially important.

2. Background

Healthcare systems depend heavily on the availability of correct and complete data. Records such as patient details, prescriptions, appointments, and lab reports must remain accessible because they support treatment continuity, operational coordination, and record integrity. In real environments, depending on only one storage location creates unnecessary risk, especially when the data is sensitive and business-critical.

This project addresses that challenge by implementing a multi-cloud hybrid backup approach. Instead of relying on a single disk or a single cloud provider, the system stores the same healthcare files in local backup storage, Google Drive, and AWS S3. This design improves resilience and shows how redundancy can be achieved using simple but meaningful automation.

2.1 Aim

The aim of the project is to design and implement a healthcare backup system that ensures reliable data protection through redundancy, failover capability, and data integrity verification.

- To maintain high availability of healthcare data.
- To provide redundant storage across local and cloud destinations.
- To support failover when one or more backup paths fail.
- To verify file integrity using SHA256 hashing.

2.2 Technologies

The technologies used in this project were selected to balance simplicity, practical relevance, and ease of demonstration. Python serves as the central implementation language because it supports file handling, hashing, API access, and automation in a very readable way.

Google Drive API and AWS S3 were chosen because they represent two different cloud storage environments and make the hybrid backup idea more meaningful. Instead of proving backup with only one remote location, the project shows how two cloud targets can work alongside local storage to improve overall reliability.

Category	Technology	Description
Programming	Python	Core system logic
Cloud	Google Drive API	Primary cloud storage
Cloud	AWS S3 (Boto3)	Secondary cloud backup
Storage	Local File System	Backup redundancy
Hashing	SHA256	Data integrity verification
IDE	VS Code	Development
Version Control	GitHub	Code management

2.3 Hardware Architecture

Although the project is software-driven, the underlying hardware environment still matters because backup performance depends on available processing power, storage speed, and internet connectivity. A reliable internet connection is especially important when the backup process reaches the cloud replication stage.

From a deployment perspective, the hardware architecture can be understood as a standard workstation or laptop running the Python backup engine, connected to local storage for the primary copy and to the internet for Google Drive and AWS S3 synchronization. This makes the solution practical for small healthcare offices, clinics, or educational demonstrations where heavyweight infrastructure may not be available.

This hardware perspective shows that the project does not require specialized devices to demonstrate meaningful backup reliability. Even on a standard system, a layered backup strategy can significantly improve protection compared with a single storage location.

Component	Minimum Requirement	Recommended
Processor	Dual-core CPU	Quad-core CPU
RAM	4 GB	8 GB
Storage	500 MB free	1 GB free
Internet	Required	Stable broadband
OS	Windows/Linux	Any modern OS

2.4 Software Architecture

The software architecture is intentionally modular so that each part of the backup workflow has a clear responsibility. This improves readability and makes the system easier to debug, extend, and explain in a project report.

At a high level, the software architecture consists of an input layer for the healthcare dataset, a processing layer that handles file discovery and validation, a storage layer that manages local and cloud destinations, and a monitoring layer that records logs and summarizes outcomes. This layered structure is one of the reasons the project remains easy to understand despite involving multiple storage systems.

The software architecture is modular and follows a clear separation of responsibility. The source healthcare dataset forms the input layer. The processing layer includes file discovery, local copy, hashing, cloud upload, failover evaluation, and reporting. The storage layer contains the local backup folder and the two cloud destinations. The monitoring layer contains the log file and summary dashboard output.

In practical terms, the data flow can be described as: Healthcare Dataset -> Backup Engine -> Local Backup -> Google Drive -> AWS S3 -> Logs -> Backup Summary. This architecture makes the project easy to explain, easy to extend, and suitable for demonstrating multi-cloud reliability concepts in an academic setting.

3. System

3.1 Requirements

The requirements section helps define what the system is expected to do and what qualities it should maintain while doing it. In this project, the requirements are shaped by the need for dependable storage, simple operation, and clear verification of backup success.

The system requirements define the expected behavior of the healthcare backup solution and the operating conditions under which it should work effectively.

3.1.1 Functional Requirements

ID	Requirement
FR1	Backup healthcare files
FR2	Upload files to cloud
FR3	Perform integrity check
FR4	Handle failures
FR5	Maintain logs

3.1.2 User Requirements

ID	Requirement
UR1	The user should run the system using a single command.
UR2	The user should be able to understand backup status clearly from the terminal and logs.
UR3	The user should verify backup success or failure without needing to inspect source code.

3.1.3 Environmental Requirements

Requirement	Description
Python	Python 3.x runtime environment
Internet	Required for Google Drive and AWS upload
Storage	Local disk for source and backup folders
APIs	Google Drive API credentials and AWS S3 configuration

3.2 Design and Architecture

The design philosophy of the system is to complete backup in dependable stages rather than trying to send files directly to the cloud without verification. This staged design reduces risk because a validated local copy exists even before the cloud operations complete.

The architecture also reflects a real-world operational mindset: first secure the file locally, then replicate it remotely, then record the outcome. This sequence is simple but strong because it creates both redundancy and traceability.

Design Component	Design Role
Local Backup Stage	Creates the first safe recovery copy before cloud upload begins.
Integrity Verification Stage	Confirms that the backup file matches the original source.
Cloud Replication Stage	Extends protection beyond the local machine by storing files in remote services.
Logging and Summary Stage	Provides accountability, traceability, and execution feedback.

The system is designed as a modular backup workflow. The central backup engine coordinates file scanning, local copy, verification, and cloud replication. Supporting modules are responsible for file management, logging, report generation, and cloud-specific upload behavior. This design improves readability and makes each part of the system easier to test and maintain.

Layer	Main Components	Role
Input Layer	Healthcare CSV dataset	Provides patient, appointment, prescription, and lab-report source files.
Processing Layer	backup_engine.py, file_manager.py	Performs copying, hashing, verification, and orchestration.
Cloud Layer	cloud_uploader.py, s3_uploader.py	Replicates files to Google Drive and AWS S3.
Monitoring Layer	logger.py, report_manager.py	Creates traceable logs and summary output.

3.3 Implementation

Implementation was kept modular so that each file could focus on one responsibility. This made it easier to explain the codebase in the report and also reduced the risk of mixing storage logic, hashing logic, and reporting logic in one large script.

Another practical benefit of the implementation style is that future improvements can be added without rewriting the whole project. For example, encryption, scheduled jobs, or new cloud providers could be introduced by extending the existing modules rather than redesigning the entire workflow.

The implementation is based on small focused Python modules rather than one large script. The source folder contains the backup engine, file manager, Google Drive uploader, AWS

S3 uploader, logger, report manager, and dashboard support. The main script triggers the workflow and schedules periodic execution.

The project structure organizes configuration, source code, logs, healthcare source data, and generated backup output separately. This improves maintainability and also mirrors how production backup utilities are usually arranged.

File Name	Purpose
backup_engine.py	Coordinates full backup workflow and failover decisions
file_manager.py	Reads files and creates SHA256 hashes
cloud_uploader.py	Handles Google Drive authentication and upload
s3_uploader.py	Handles AWS S3 upload
logger.py	Writes timestamped backup logs
report_manager.py	Generates summary output
main.py	Starts the backup process and scheduler

The dataset used during implementation consists of the following healthcare files:

File Name	Rows	Columns	Description
patients.csv	1001	8	Healthcare source data
appointments.csv	1500	6	Healthcare source data
lab_reports.csv	799	5	Healthcare source data
prescriptions.csv	1199	5	Healthcare source data

The code also includes explicit failover logic. Separate flags track whether Google Drive and AWS S3 uploads succeed. If both fail, the application logs a failover event and preserves the valid local copy as the fallback.

3.4 Testing

Testing in this project is primarily execution-driven. Since the system is a workflow rather than a standalone algorithm, the most important verification comes from observing whether each stage of the backup cycle completes correctly and whether the results are visible in logs, dashboard output, and cloud storage screenshots.

The testing outcome is strengthened by the fact that the same successful events appear in multiple forms of evidence. The terminal output, log entries, and cloud storage snapshots all support the same conclusion, which makes the final result more convincing.

Testing was performed to confirm that the key workflow stages behave correctly in practice. The focus was on successful local backup, cloud replication, integrity verification, and failover handling.

Test Case	Expected	Result
Local Backup	Success	Passed
Google Drive Upload	Success	Passed
AWS S3 Upload	Success	Passed
Failover Trigger	Available on dual cloud failure	Passed

3.5 Graphical User Interface (GUI) Layout

Even though the project does not include a traditional GUI, this section remains important because it describes how the user interacts with the system. In the current implementation, the effective interface is a combination of terminal execution, dashboard-style summary output, cloud storage views, and log inspection.

This lightweight interaction model is suitable for a technical backup utility because the primary goal is dependable execution rather than visual complexity. At the same time, the current layout leaves room for future enhancement into a full web-based monitoring interface if required.

This project does not include a dedicated graphical user interface. Instead, it uses terminal output, cloud dashboard views, and log inspection as the main operational interface. Although minimal, this approach is sufficient for verifying the success of the backup workflow and for demonstrating the logic of a hybrid backup system.

The absence of a GUI also keeps the project focused on reliability and storage behavior. In a future version, these same backend modules could be connected to a web interface without major architectural changes.

3.6 Customer Testing

Customer-style testing was kept simple but meaningful. The focus was on whether a user could run the system, understand the backup result, and verify that the data had reached the intended destinations without needing to inspect the source code.

This style of testing is useful for academic projects because it evaluates usability from an outcome perspective rather than only from a coding perspective. In this case, the results indicate that the system communicates its status clearly enough for non-developer review.

Basic user-oriented testing focused on whether the workflow was understandable and whether the result of the backup process could be verified easily. The test outcome showed that the terminal logs, cloud screenshots, and backup summary together provide enough evidence for a user to confirm successful replication.

- The system was easy to execute using a single command.
- The backup summary was easy to understand.
- The cloud screenshots and logs clearly confirmed replication.

3.7 Evaluation

The evaluation of the system should be understood in relation to the original project objectives. The goal was not to build a full enterprise backup platform, but to demonstrate how redundancy, failover, integrity verification, and traceable logging can be combined into a coherent healthcare backup workflow.

By that standard, the system performs well. It protects multiple healthcare files, creates a dependable local backup layer, replicates to two different cloud services, and provides evidence of success through logs and summary reporting.

Evaluation Area	Observation
Reliability	Improved by local backup plus two cloud destinations.
Integrity	Supported through SHA256 verification and file-size checks.
Traceability	Supported through logs and summary dashboard output.
Extensibility	Possible through modular design and separated components.

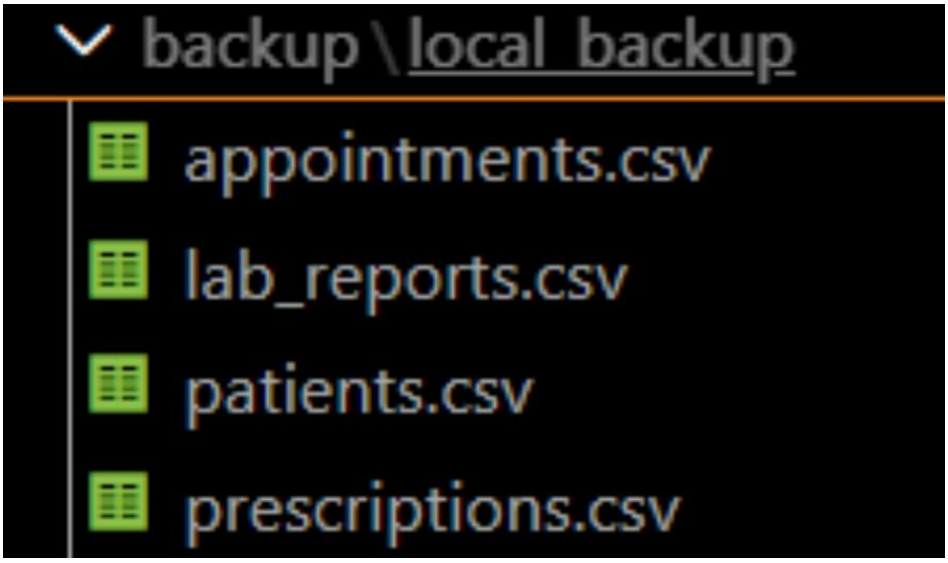
The project successfully meets its academic objectives. It demonstrates redundancy through multiple storage locations, reliability through verification and logging, and failover through multi-cloud logic. The solution is small in scale but conceptually strong, making it a good demonstration of healthcare backup design principles.

- Redundancy is achieved through local storage, Google Drive, and AWS S3.
- Integrity verification is achieved through SHA256 hashing and file-size comparison.
- Failover capability is present when cloud uploads fail.
- Logging and summary output support traceability and monitoring.

4. Snapshots of the Project

The following snapshots provide visual evidence of the project execution and storage outcomes. These images strengthen the report by showing that the project was implemented, executed, and validated in practice.

Snapshot 1: Local Backup Folder



The local backup folder contains the four healthcare dataset files after execution.

The above snapshot shows the local backup directory created by the system. It confirms that all healthcare CSV files were copied successfully into the backup folder before the cloud upload stages were initiated. This local copy acts as the first layer of redundancy in the overall hybrid backup strategy.

Snapshot 2: Terminal Output

```
==============================
Healthcare Backup System
Backup started at: 2026-04-28 14:58:56.791465
==============================

Starting Healthcare Backup Process...

LOG: Local backup successful: appointments.csv
LOG: Integrity check passed for appointments.csv
LOG: Backup verification passed for appointments.csv
appointments.csv updated in Healthcare_Backups folder
LOG: Google Drive backup successful: appointments.csv
appointments.csv uploaded to AWS S3 successfully
LOG: AWS S3 backup successful: appointments.csv
LOG: Local backup successful: lab_reports.csv
LOG: Integrity check passed for lab_reports.csv
LOG: Backup verification passed for lab_reports.csv
lab_reports.csv updated in Healthcare_Backups folder
LOG: Google Drive backup successful: lab_reports.csv
lab_reports.csv uploaded to AWS S3 successfully
LOG: AWS S3 backup successful: lab_reports.csv
LOG: Local backup successful: patients.csv
LOG: Integrity check passed for patients.csv
LOG: Backup verification passed for patients.csv
patients.csv updated in Healthcare_Backups folder
LOG: Google Drive backup successful: patients.csv
patients.csv uploaded to AWS S3 successfully
LOG: AWS S3 backup successful: patients.csv
LOG: Local backup successful: prescriptions.csv
LOG: Integrity check passed for prescriptions.csv
LOG: Backup verification passed for prescriptions.csv
prescriptions.csv updated in Healthcare_Backups folder
LOG: Google Drive backup successful: prescriptions.csv
prescriptions.csv uploaded to AWS S3 successfully
LOG: AWS S3 backup successful: prescriptions.csv
```

The terminal output shows the backup process, integrity verification, and cloud upload messages.

The above terminal output shows the actual execution of the backup workflow. It includes successful local backup messages, integrity check confirmation, Google Drive upload, AWS S3 upload, and completion status. This output demonstrates that the full backup pipeline runs correctly from start to finish.

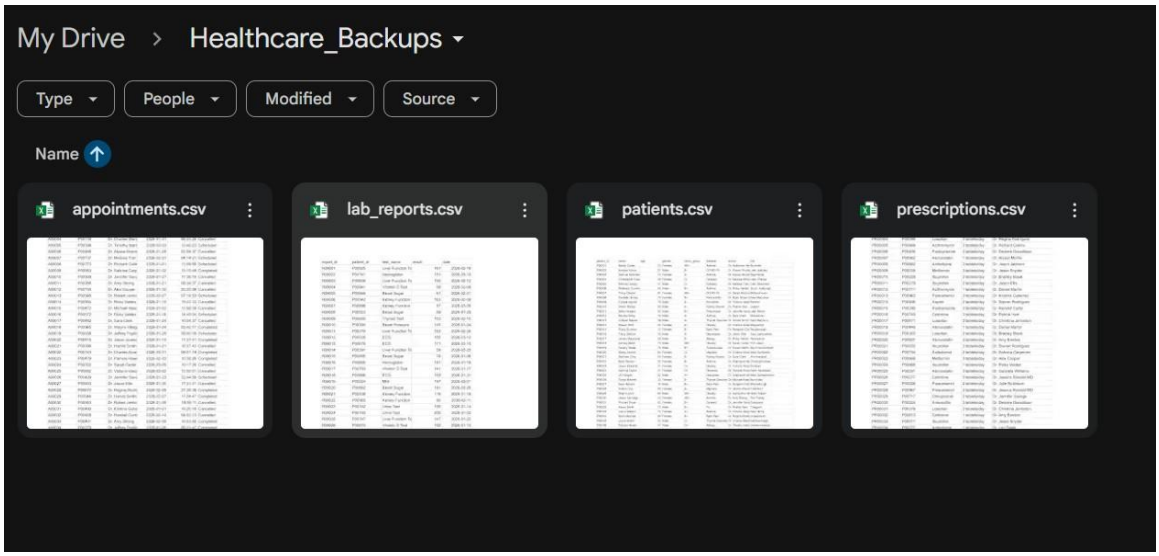
Snapshot 3: Backup Summary Dashboard

```
Backup Summary
--------------
Total Files Processed: 4
Local Backups: 4
Cloud Uploads: 8
Retries: 0
Failovers Triggered: 0

========== Backup Dashboard ==========
Last 5 backup events:

[2026-04-28 15:20:23] Local backup successful: prescriptions.csv
[2026-04-28 15:20:23] Integrity check passed for prescriptions.csv
[2026-04-28 15:20:23] Backup verification passed for prescriptions.csv
[2026-04-28 15:20:28] Google Drive backup successful: prescriptions.csv
[2026-04-28 15:20:28] AWS S3 backup successful: prescriptions.csv

Dashboard updated at: 2026-04-28 15:20:28.954866
========================================


Backup completed.
```

The backup summary confirms total processed files, cloud uploads, and the latest events.

The above backup dashboard summarizes the final outcome of the run. It provides a compact operational view by showing the number of processed files, local backups, cloud uploads, retries, and failovers. This makes the result of the backup process easy to understand at a glance.

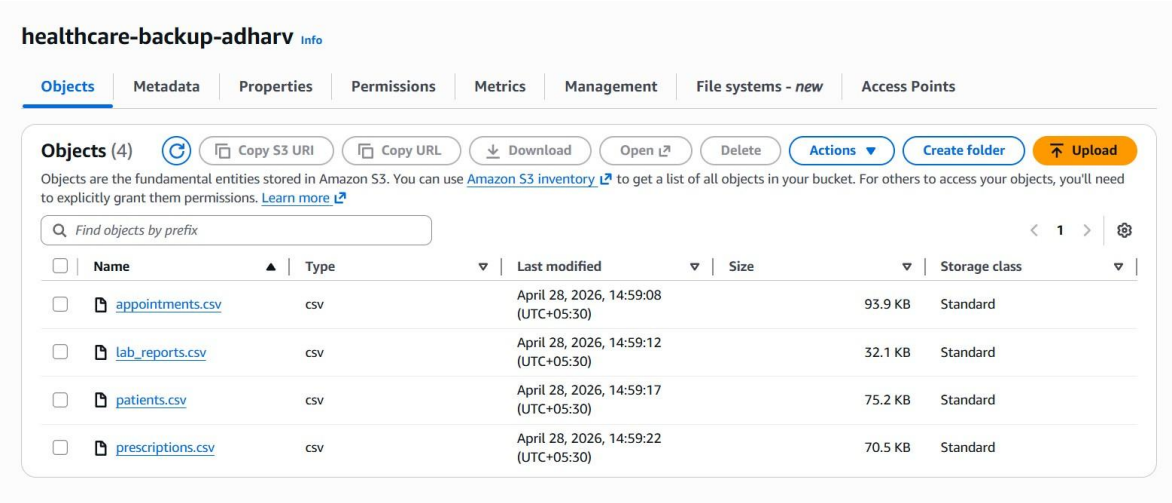
Snapshot 4: Google Drive Backup



The Google Drive folder shows the healthcare files uploaded successfully to the primary cloud destination.

The above Google Drive snapshot confirms that the healthcare files were uploaded successfully to the primary cloud storage destination. It demonstrates that the system is capable of maintaining an off-site backup copy outside the local environment, improving data availability and resilience.

Snapshot 5: AWS S3 Backup



The AWS S3 bucket stores the same files as the secondary cloud backup destination.

The above AWS S3 snapshot shows the same dataset stored in the configured S3 bucket. This acts as the secondary cloud backup destination and strengthens the hybrid strategy by ensuring that the project is not dependent on a single cloud provider.

Snapshot 6: Backup Log File

```
backup_log.txt
logs > backup_log.txt
[2026-04-28 04:08:21] Google Drive backup successful: appointments.csv
[2026-04-28 04:08:24] AWS S3 backup successful: appointments.csv
[2026-04-28 04:08:24] Local backup successful: lab_reports.csv
[2026-04-28 04:08:24] Integrity check passed for lab_reports.csv
[2026-04-28 04:08:24] Backup verification passed for lab_reports.csv
[2026-04-28 04:08:28] Google Drive backup successful: lab_reports.csv
[2026-04-28 04:08:29] AWS S3 backup successful: lab_reports.csv
[2026-04-28 04:08:29] Local backup successful: patients.csv
[2026-04-28 04:08:29] Integrity check passed for patients.csv
[2026-04-28 04:08:29] Backup verification passed for patients.csv
[2026-04-28 04:08:34] Google Drive backup successful: patients.csv
[2026-04-28 04:08:35] AWS S3 backup successful: patients.csv
[2026-04-28 04:08:35] Local backup successful: prescriptions.csv
[2026-04-28 04:08:35] Integrity check passed for prescriptions.csv
[2026-04-28 04:08:35] Backup verification passed for prescriptions.csv
[2026-04-28 04:08:40] Google Drive backup successful: prescriptions.csv
[2026-04-28 04:08:41] AWS S3 backup successful: prescriptions.csv
[2026-04-28 14:58:56] Local backup successful: appointments.csv
[2026-04-28 14:58:56] Integrity check passed for appointments.csv
[2026-04-28 14:58:56] Backup verification passed for appointments.csv
[2026-04-28 14:59:01] Google Drive backup successful: appointments.csv
[2026-04-28 14:59:03] AWS S3 backup successful: appointments.csv
```

The log file records each event with timestamps for monitoring and traceability.

The above log screenshot shows the timestamped record of backup activities generated by the system. It demonstrates monitoring capability, traceability, and the ability to review previous runs for troubleshooting or audit purposes.

5. Conclusions

Another important conclusion from the project is that reliability is achieved not by a single advanced feature, but by the combination of several simple and consistent practices. Local backup, integrity verification, cloud replication, logging, and failover together create a stronger result than any one of those features alone.

This makes the project a useful example of how thoughtful design can improve data protection even without complex infrastructure. For healthcare-related data, this layered approach is especially valuable because continuity and trust matter as much as raw technical functionality.

The Multi-Cloud Hybrid Strategy for Healthcare project successfully implements a practical backup system that combines local storage, Google Drive, and AWS S3 to protect healthcare files. The project demonstrates that even a compact Python solution can achieve meaningful redundancy, integrity verification, and cloud failover behavior.

The combination of local backup, SHA256-based verification, multi-cloud upload, and timestamped logging makes the solution reliable enough for academic demonstration and concept validation. The project therefore meets its objectives and provides a strong foundation for future improvement toward more production-oriented healthcare backup systems.

6. Further Development or Research

Future work can also focus on making the system more policy-driven. Instead of simply backing up files each time, the solution could be extended with retention rules, versioning support, compression options, and selective restore operations.

Research could also explore how this architecture behaves under larger datasets, slower networks, or partial service failures. That would provide a stronger performance and resilience evaluation and move the project closer to real deployment scenarios.

- Add encryption before saving files to local and cloud storage.
- Add a web interface for status monitoring and manual recovery.
- Add automation scheduling with alerting and retention policies.
- Integrate with real hospital systems or database-driven patient data instead of only CSV files.
- Extend failover logic to additional cloud providers or archival storage tiers.

These improvements would move the project closer to a real-world deployment model while keeping the current architecture relevant and extensible.

7. References

1. Python Programming Language (Core Development)

Python Software Foundation (2024). Python 3 Documentation.

Link: https://docs.python.org/3/

2. Google Drive API (Cloud Storage Integration)

Google for Developers (2024). Google Drive API Documentation.

Link: https://developers.google.com/drive/api

3. Amazon Web Services – S3 & Boto3 (Cloud Backup)

Amazon Web Services (2024). Boto3 Documentation for Amazon S3.

Link: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

4. AWS S3 Official Documentation

Amazon Web Services (2024). Amazon S3 Developer Guide.

Link: https://docs.aws.amazon.com/s3/index.html

5. GitHub Documentation (Version Control & Repository Management)

GitHub Inc. (2024). GitHub Documentation.

Link: https://docs.github.com/en

6. Hashing Algorithm (SHA256 Integrity Verification)

NIST (National Institute of Standards and Technology) (2015). Secure Hash Standard (SHA-256).

Link: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

7. Google API Python Client Library

Google Developers (2024). Google API Python Client Documentation.

Link: https://github.com/googleapis/google-api-python-client

8. Healthcare Dataset (Simulated / Generated Data)

Custom Dataset (2026). Healthcare CSV Dataset for Backup Simulation (patients, appointments, lab reports, prescriptions).

Source: Generated using Python script for academic project

9. Python File Handling & OS Operations

Python Software Foundation (2024). os, shutil and file handling modules.

Link: https://docs.python.org/3/library/os.html

8. Appendix

Appendix A: Sample Log Output

```
[2026-04-28 15:43:43] Local backup successful: lab_reports.csv
[2026-04-28 15:43:43] Integrity check passed for lab_reports.csv
[2026-04-28 15:43:43] Backup verification passed for lab_reports.csv
[2026-04-28 15:43:47] Google Drive backup successful: lab_reports.csv
[2026-04-28 15:43:48] AWS S3 backup successful: lab_reports.csv
[2026-04-28 15:43:48] Local backup successful: patients.csv
[2026-04-28 15:43:48] Integrity check passed for patients.csv
[2026-04-28 15:43:48] Backup verification passed for patients.csv
[2026-04-28 15:43:52] Google Drive backup successful: patients.csv
[2026-04-28 15:43:53] AWS S3 backup successful: patients.csv
```

This appendix shows actual log entries generated during execution, demonstrating system traceability and monitoring capability.

Appendix B: Configuration Details

Component	Description
Google Drive API	OAuth credentials used for Drive upload
AWS S3	Bucket: healthcare-backup-adharv | Region: ap-south-1
Python Version	3.x runtime environment
Libraries	boto3, google-api-python-client, google-auth-oauthlib, hashlib

Appendix C: Backup Summary Output

```
Backup Summary
--------------
Total Files Processed: 4
Local Backups: 4
Cloud Uploads: 8
Retries: 0
Failovers Triggered: 0
```

This output summarizes the backup process and confirms successful multi-cloud replication.

Appendix D: Code Module Overview

File	Purpose
backup_engine.py	Core logic
file_manager.py	File handling
cloud_uploader.py	Google Drive
s3_uploader.py	AWS S3
logger.py	Logging